Chargeback Risk Predictor

An AI-based risk detector for merchant chargebacks

AI Risk Manager — Fraud & Chargeback Risk

Project Report

Kriti Goel
B.Tech Electrical Engineering, NSUT Delhi

1. Project Overview

1.1 What this project is

Chargeback Risk Predictor is a machine learning system that predicts whether a transaction is likely to result in a chargeback, explains why, and recommends an appropriate action.

1.2 The problem it solves

Merchants using payment gateways lose money to fraud, returns, and chargebacks. A chargeback happens when a customer disputes a transaction with their bank, forcing a forced refund — often with an additional penalty to the merchant. Left undetected, high chargeback rates can also lead to a merchant's payment account being flagged or suspended.

The goal was to build a working detector for one class of loss — with measured precision and recall on a held-out test set, and honest reporting of false-positive cost — rather than a purely theoretical model.

1.3 Why this problem was chosen

Three loss types were considered for this track: fraud, return abuse, and chargebacks. Fraud has the most public data available but is the most commonly picked (highest risk of overlap with other submissions). Return abuse requires inventing subjective abuse-rules with a risk of circular logic (rules define the data, data confirms the rules). Chargeback risk was chosen because:

- It is directly relevant to any payment platform's core business — every merchant accepting online payments is exposed to chargeback risk.
- Risk signals are relatively objective (transaction amount, address mismatch, account age, delivery evidence) rather than requiring invented behavioural rules.
- It naturally extends into an explainable, actionable system — not just a yes/no flag, but a reason and a suggested next step.

2. Approach

2.1 High-level pipeline

Synthetic transaction data (with business-rule-based chargeback outcomes) -> Exploratory data analysis and edge-case validation -> Model training (Logistic Regression baseline, then Random Forest) -> Threshold tuning -> Cost-impact analysis -> Edge-case scenario testing -> Reason-classification layer -> Interactive dashboard.

2.2 Why synthetic data

Real chargeback-labelled datasets are not publicly available (they involve sensitive customer and payment data). Public fraud datasets that do exist (e.g. anonymised credit-card fraud data) use PCA-transformed, unlabelled columns, which would have made it impossible to build the

business-context features (sale-period effects, salary-week effects, delivery evidence strength) that are central to this project's approach. A synthetic dataset, generated from explicit, documented business rules, was used instead.

2.3 Features used (20 total)

Features were grouped into six categories:

- Core transaction signals: amount, hour of day, payment method
- Customer history signals: account age, average order value, past chargeback count
- Mismatch signals: billing/shipping mismatch, cardholder name mismatch score, device-sharing flag
- Velocity signals: transactions in last 24h, failed attempts before success, checkout speed
- Verification/evidence signals: secondary (OTP/bank-call) verification, delivery confirmation strength, refund-requested-before-chargeback, duplicate-transaction flag
- Context flags (the differentiating idea of this project): sale-period, salary-week, high-value category (medical/travel), recent location change

The context flags exist to prevent a naive model from flagging normal Indian consumer behaviour as fraud — for example, a large purchase right after payday, a fast checkout during a flash sale, or a billing/shipping mismatch caused by genuine travel. Each of these was explicitly designed as an interaction effect (e.g. "fast checkout is only suspicious if it is NOT a sale period") rather than a flat rule.

3. Data Generation and Validation

5,000 synthetic transactions were generated in Python (pandas/numpy) using explicit business rules for each feature, with random noise added so the data is not perfectly separable (reflecting real-world messiness). The resulting chargeback rate is 16.18%, intentionally higher than the real-world industry rate of roughly 0.5-1%, to ensure the model sees enough positive examples to learn from in a 5,000-row dataset.

Before training any model, four context-based edge cases were explicitly validated against the generated data, to confirm the intended business logic actually held:

```
========================================================================
PART B: EDGE CASE VALIDATION
========================================================================

[Test 1] Fast checkout (<10s) chargeback rate:
  During SALE period     : 15.57%
  During NON-sale period : 26.85%
  --> Non-sale should be noticeably higher

[Test 2] High amount (>20k) chargeback rate:
  During SALARY week     : 16.67%
  During NON-salary week : 50.00%
  --> Non-salary-week should be noticeably higher

[Test 3] Duplicate transaction flag:
  Chargeback rate WHEN duplicate=1 : 50.99%
  Chargeback rate WHEN duplicate=0 : 15.10%
  Reason breakdown for duplicate chargebacks:
chargeback_reason
Accidental-Duplicate    77
Name: count, dtype: int64

[Test 4] Billing/shipping mismatch chargeback rate:
  WITH recent travel    : 12.12%
  WITHOUT recent travel : 23.87%
  --> Without travel should be higher
```

Figure 1 — Edge-case validation: chargeback rate consistently drops when a risky-looking signal is explained by context (sale period, salary week, travel), and rises sharply for duplicate transactions.

General exploratory analysis was also run: missing values, feature distributions, correlations with the chargeback outcome, and category-wise breakdowns (payment method, delivery type, high-value category).

```
PART A: GENERAL EDA
========================================================================

[Overall] Chargeback rate: 16.18%

[Missing values check]
0 missing values total

[Numeric feature summary]
         amount  account_age_days  avg_order_value  checkout_speed_sec  transactions_last_24h  name_mismatch_score
count    5000.00           5000.00          5000.00             5000.00                5000.00              5000.00
mean     1771.01            295.31           672.11               44.99                   1.20                 0.11
std      4157.61            295.12           651.49               46.41                   1.12                 0.10
min       100.00              0.00            13.70                0.00                   0.00                 0.00
25%       264.79             85.00           286.28               12.60                   0.00                 0.03
50%       677.80            203.00           485.87               30.30                   1.00                 0.08
75%      1689.88            402.00           837.05               61.60                   2.00                 0.16
max    100000.00           2428.00         13606.68              408.90                   7.00                 0.62

[Correlation with is_chargeback] (numeric features only)
is_chargeback                      1.000000
duplicate_transaction_flag         0.166819
past_chargeback_count              0.128336
refund_requested_before_chargeback 0.118254
amount                             0.101311
failed_attempts_before_success     0.074492
device_sharing_flag                0.068291
billing_shipping_mismatch          0.067584
secondary_verification_flag        0.062567
name mismatch score                0.037823
```

Figure 2 — General EDA: 0 missing values, sensible correlations (duplicate_transaction_flag, past_chargeback_count and refund_requested_before_chargeback rank highest).

4. Model Training and Evaluation

4.1 Models tried

A Logistic Regression baseline was trained first (simple, explainable, class_weight='balanced' to account for the ~16% minority class). A Random Forest was then trained on the same 80/20 train-test split for comparison.

4.2 Final results (Random Forest, threshold = 0.5)

Metric	Value
Precision	40.0%
Recall	38.3%
F1 Score	39.1%
ROC-AUC	0.749
Chargeback rate (test set)	16.2%

```
======================================================================
RANDOM FOREST — Results
======================================================================

Confusion Matrix: TN=745 FP=93 FN=100 TP=62
Precision : 40.00%
Recall    : 38.27%
F1 Score  : 39.12%
ROC-AUC   : 0.749

======================================================================
COMPARISON: Logistic Regression (baseline) vs Random Forest
======================================================================
Metric          LogReg  RandForest
Precision       20.39%      40.00%
Recall          62.71%      38.27%
F1 Score        30.77%      39.12%
ROC-AUC          0.687       0.749

[Top 10 most important features]
amount                                          0.1330
checkout_speed_sec                              0.1230
avg_order_value                                 0.1020
account_age_days                                0.0996
duplicate_transaction_flag                      0.0648
refund_requested_before_chargeback              0.0614
name_mismatch_score                             0.0612
hour_of_day                                     0.0602
delivery_confirmation_strength_otp_confirmed    0.0477
past_chargeback_count                           0.0432
dtype: float64

Saved Random Forest model and predictions.
```

Figure 3 — Random Forest results and top feature importances (amount, checkout speed, average order value, account age, and duplicate-transaction flag rank highest).

```
======================================================================
STEP 4: MODEL EVALUATION (Baseline — Logistic Regression)
======================================================================

[Confusion Matrix]
                    Predicted: No CB   Predicted: CB
  Actual: No CB            594               244   <- False Positives (FP)
  Actual: CB                64                98   <- True Positives (TP)
                          ^False Negatives (FN)

  True Negatives  (TN): 594  -- correctly said 'safe'
  False Positives (FP): 244  -- WRONGLY flagged a genuine customer
  False Negatives (FN): 64  -- MISSED an actual chargeback
  True Positives  (TP): 98  -- correctly caught a chargeback

[Core Metrics]
  Precision : 28.65%  -- of everyone we flagged, how many were ACTUALLY risky
  Recall    : 60.49%  -- of all ACTUAL chargebacks, how many did we catch
  F1 Score  : 38.89%  -- balance between precision and recall
  ROC-AUC   : 0.729  -- overall ability to rank risky vs safe (0.5=random, 1.0=perfect)

[Full classification report]
               precision    recall  f1-score   support

No Chargeback       0.90      0.71      0.79       838
   Chargeback       0.29      0.60      0.39       162

     accuracy                           0.69      1000
    macro avg       0.59      0.66      0.59      1000
 weighted avg       0.80      0.69      0.73      1000

Saved confusion matrix for cost analysis step.
```

Figure 4 — Confusion matrix and full classification report on the 1,000-row held-out test set.

4.3 Why Random Forest was chosen over Logistic Regression

Logistic Regression achieved higher recall (catches more chargebacks) but much lower precision (20-29%, meaning most flagged transactions were false alarms). Random Forest achieved higher precision (40%) at a moderate recall (38%). Since a payment platform's priority is protecting merchant checkout conversion and customer trust — not absorbing the fraud loss directly — minimising false positives (genuine customers being wrongly flagged) was treated as the higher priority. Random Forest, at threshold 0.5, was chosen as the final model on this basis.

4.4 Threshold tuning

Multiple thresholds (0.2 to 0.7) were tested to understand the precision-recall trade-off. No threshold was found that improves both precision and recall simultaneously — this is documented as a known limitation (Section 7), not hidden. Threshold 0.5 was selected as the best balance between catching real chargebacks and avoiding customer friction, in line with the platform-priority reasoning above.

5. Cost Impact Analysis

To translate model performance into a business result, the confusion matrix was converted into an illustrative amount estimate, using assumed figures (average order value Rs.3,000, friction cost per false positive Rs.150 — both estimates, not real merchant data, and explicitly labelled as such).

Item	Amount
Value from 62 chargebacks caught early	+ Rs. 1,86,000
Cost of 93 false positives (review friction)	- Rs. 13,950
Net value added by the model	Rs. 1,72,050

The 100 chargebacks still missed by the model (Rs. 3,00,000) are not counted as a cost caused by the model — that loss would occur with or without any detection system in place. They represent room for future improvement rather than a model weakness to subtract from its value.

6. Edge-Case Testing and the Reason-Classification Layer

6.1 Edge-case scenario testing

Beyond aggregate metrics, the final model was tested on hand-built realistic scenarios to check it behaves sensibly on cases a reviewer might specifically ask about — e.g. a fast checkout during a festive sale, a big purchase during salary week, a new customer buying for a medical emergency, and an address mismatch explained by recent travel. In each context-aware pair, the model's risk score moved in the expected direction (lower when context explains the signal, higher without it).

6.2 Reason-classification layer (Stage 2)

A transaction flagged as risky is not, by itself, actionable for a merchant. A second, rule-based layer classifies WHY a flagged transaction is risky, into one of four categories — Fraud-Suspected, Accidental-Duplicate, Service-Issue, or Genuine-Dispute — and attaches a suggested action for each (e.g. auto-refund a duplicate immediately vs freeze-and-investigate a suspected fraud). This was deliberately built as an explainable rule-based layer rather than a second ML classifier, since some reason categories had too few examples in the 5,000-row dataset to train a reliable classifier on, and a transparent, auditable decision layer is more appropriate for a financial risk system in any case.

```
================================================================================
STEP 7: RISK + REASON + SUGGESTED ACTION (full pipeline demo)
================================================================================

[Duplicate payment from a loyal customer]
  Risk score       : 33.17%
  Flagged          : False

[New account, stolen-card-like pattern]
  Risk score       : 53.50%
  Flagged          : True
  Reason           : Fraud-Suspected
  Suggested action : Freeze/hold transaction, compile evidence (device, address, ID mismatch), route to fraud review.

[Refund denied, escalated to bank]
  Risk score       : 54.27%
  Flagged          : True
  Reason           : Service-Issue
  Suggested action : Escalate to customer support -- check why the earlier refund request was not honored.
```

Figure 5 — Full pipeline demo: risk score, flag decision, reason, and suggested action for three example transactions.

7. Known Limitations (Build Challenges)

These are stated openly, as required by the track's evaluation bar ("honest metrics including false-positive cost"):

- Precision (40%) and recall (38%) leave room to grow. Roughly 6 in 10 flagged transactions turn out safe, and about 6 in 10 real chargebacks are missed. Threshold tuning found no single setting that improves both simultaneously — this trade-off is inherent to the current feature set, not a tuning oversight.
- Data-generation noise vs. signal trade-off: the chargeback rate was initially compressed from ~16% to ~12% using a uniform scaling factor plus added noise, to look more "realistic". This was later diagnosed as the root cause of weak model performance (noise was comparable in size to several rule effects, blurring the learnable signal). Removing the scaling and reducing noise improved Random Forest precision from 31% to 40% and ROC-AUC from 0.688 to 0.749. A second attempt (tightening trigger thresholds instead of scaling) was also tested and performed worse, confirming that the original, unscaled ~16% rate was in fact the best-performing configuration for this dataset design — a deliberate, tested trade-off, not an oversight.
- The duplicate-transaction signal is under-weighted relative to its true strength in the data, because duplicates are naturally rare (~3% of transactions). This was confirmed to be a data-volume issue and not a logic issue: temporarily increasing duplicate frequency to 7% moved this feature from importance rank #10 to rank #1. The change was rolled back because it slightly reduced overall precision/recall, and the trade-off was judged not worth it — but the diagnosis itself is documented here as a validated finding.
- All results are based on synthetic data with hand-designed rules, not real transaction data from any payment platform. Feature relationships (sale-period effect, salary-week effect, etc.) were built into the data by design. A production version would need to learn these relationships from real, unlabelled historical data instead of having them assumed in advance.
- The assumed rupee figures in the cost analysis (Rs.3,000 average order value, Rs.150 friction cost per false positive) are illustrative estimates, not real merchant figures, and are explicitly labelled as such throughout.

8. Interactive Dashboard

A standalone HTML dashboard (dashboard.html, included in the repository) presents the full results as an interactive, presentable interface: headline business impact, model performance metrics, feature importance, a live "try a transaction" explorer using nine real curated scenarios (with real model predictions, not simulated), a chargeback-reason breakdown, and the context-awareness comparisons described in Section 3. It opens directly in any browser with no server or installation required.

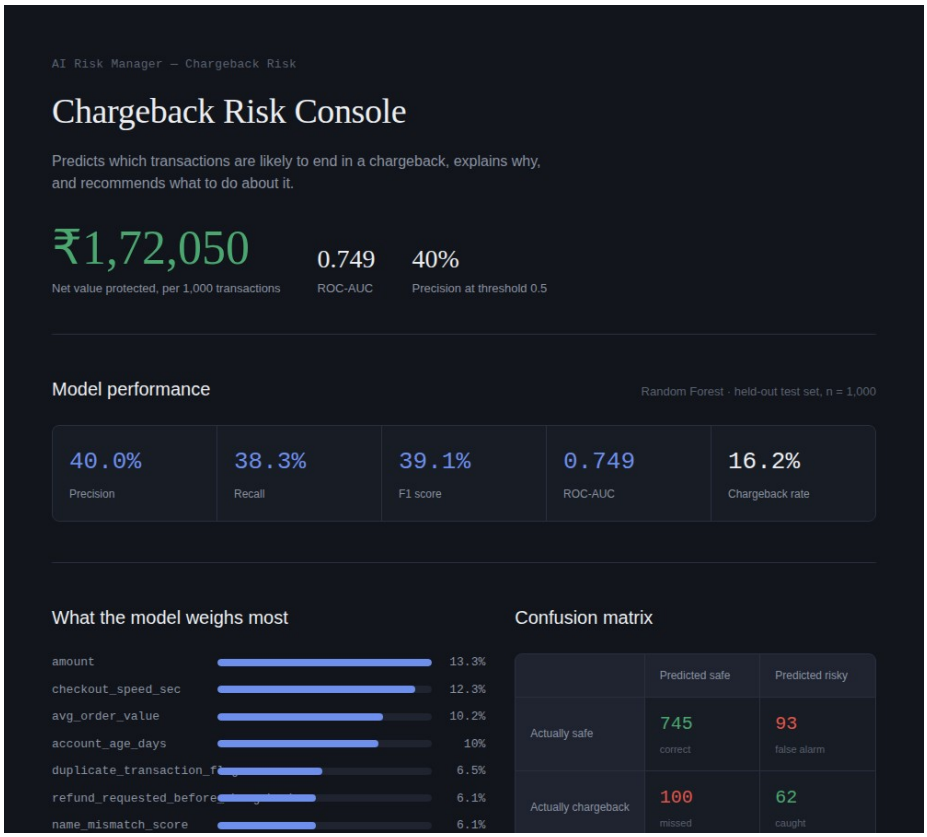

Figure 11 — Dashboard hero and top metrics view.

9. Supplementary Visual Analysis (Power BI)

The following charts were built in Power BI directly from the generated transaction dataset (transactions.csv), to visually confirm the findings discussed above.

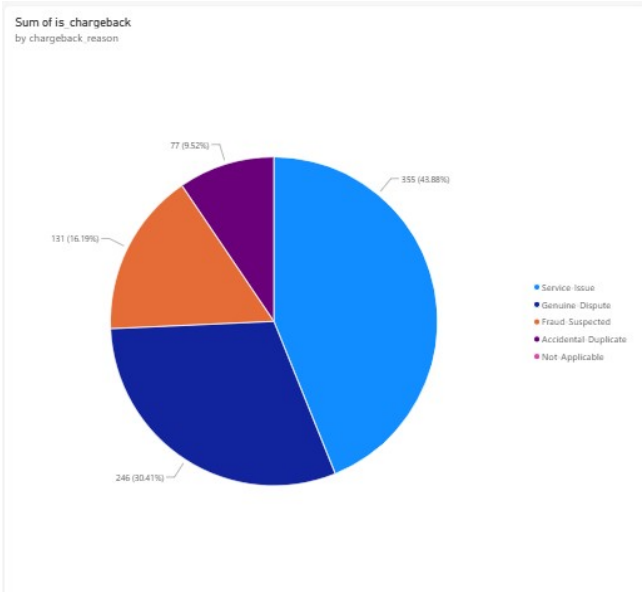

Figure 6 — Chargeback reasons: Service-Issue (43.9%) and Genuine-Dispute (30.4%) dominate — chargebacks are not primarily a fraud problem.

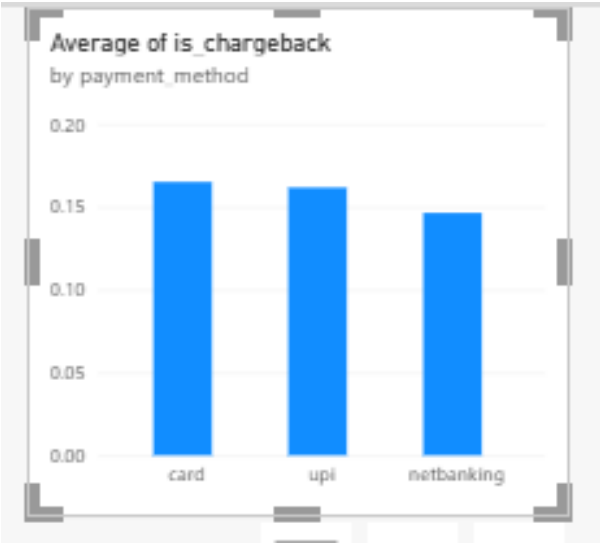

Figure 7 — Chargeback rate by payment method: card and UPI carry marginally higher risk than netbanking.

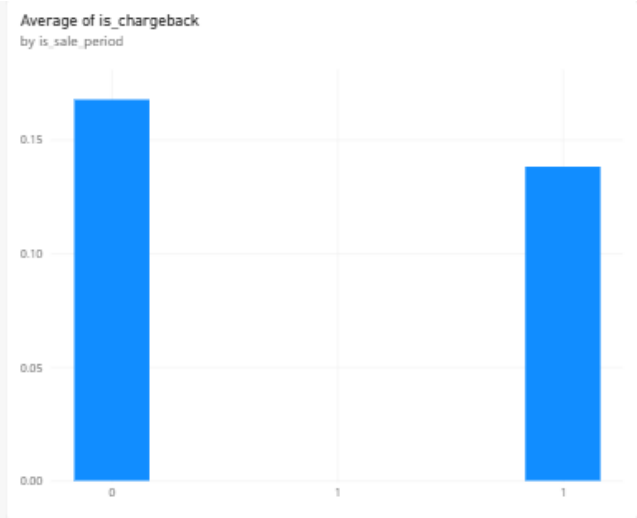

Figure 8 — Fast-checkout chargeback rate is lower during a sale period than on an ordinary day, confirming the context-awareness rule.

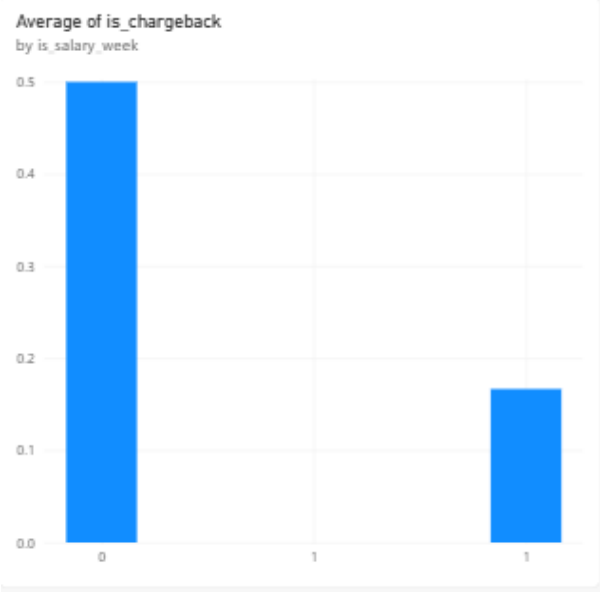

Figure 9 — Among high-amount (>Rs.20,000) transactions, chargeback rate drops sharply during salary week versus other days.

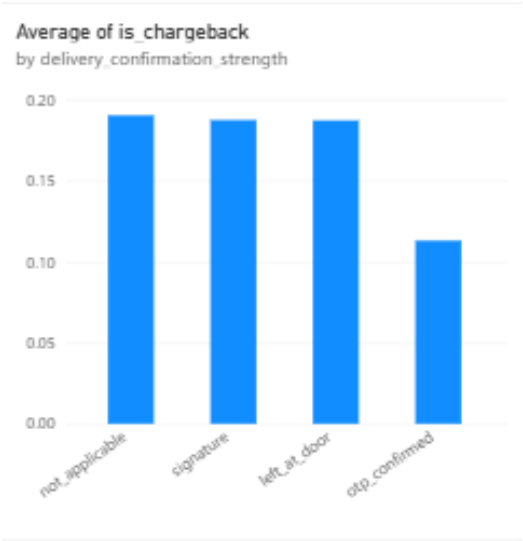

Figure 10 — OTP-confirmed delivery shows the lowest chargeback rate of all delivery-confirmation types, supporting its use as a protective signal.

10. Tools and Technologies Used

- Python — pandas, numpy (synthetic data generation and analysis)
- scikit-learn — Logistic Regression, Random Forest, train/test split, metrics
- joblib — model persistence
- HTML/CSS/JavaScript — interactive standalone dashboard
- Power BI — supplementary business-intelligence visualisations
- Google Colab — execution environment for reproducing all results in this report

11. Conclusion

This project delivers a working chargeback-risk detector that goes beyond a binary flag: it explains its reasoning, recommends an action, quantifies its own business value in rupee terms, and documents its own limitations honestly and with evidence. Every design decision in this report — from the choice of chargebacks over fraud or returns, to the choice of Random Forest over Logistic Regression, to the decision to accept a ~16% synthetic chargeback rate after testing alternatives — was made deliberately and is backed by a specific, reproducible test recorded in this document.